

SIMATS ENGINEERING

ASSESSMENT TOOL II — DATA ANALYSIS PROJECT

Course: Data Handling and Visualization	Code: DSA0603	CO: CO3
Duration: 180 min	Total Marks: 100	Reg No : 192424303 Name: P.Sunayana

COURSE OUTCOME COVERED

CO3: Analyze time-series data, trends, associations, and uncertainty through appropriate visualization methods. (BL4)

Activity Tool : Data Analysis Project

Weightage: 20%

Code of Conduct:

I P.Sunayana 192424303 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Sunayana

Criteria	Understanding of Concepts 25%	Analytical Design 25%	Use of Tools 20%	Analysis of Results 20%	Documentation 10%	Total
Q1						
Q2						
Q3						
Q4						
Q5						

DSA0603 — DATA HANDLING AND VISUALIZATION

CO3_AT2_Assessment Tool: DATA ANALYSIS PROJECT

Topics Covered: ECDF and Q-Q Plots; Multiple Distributions; Vertical-Axis Distributions; Bean Plots; Proportions; Pie/Side-by-Side/Stacked Bars; Stacked Densities; Treemaps; Sunburst Charts; Parallel Sets; Sankey Diagrams; Nested Proportions; Avoiding Misleading Proportions.

General Instruction: For each project task, use an appropriate visualization in R, justify the design choice, interpret the result in context, and identify potential visualization bias. R/ggplot2 (or an equivalent tool such as treemapify, ggalluvial, or networkD3) may be used. Submit R code, output plots, and a short written interpretation for each question.

Question 1 — Comparing Distributions with ECDF and Q-Q Plots

Scenario: A logistics company records delivery time (hours) for two courier partners over 20 shipments each. Operations wants to know whether one partner is consistently faster and whether either partner's delivery times follow a normal pattern that would justify simple mean-based SLAs.

Dataset:

Partner	Delivery Times (hours), n = 20
Partner A	18, 19, 20, 20, 21, 21, 22, 22, 23, 23, 24, 24, 25, 26, 27, 28, 30, 33, 38, 46
Partner B	20, 21, 22, 22, 23, 23, 23, 24, 24, 24, 24, 25, 25, 25, 26, 26, 27, 27, 28, 29

Tasks:

1. Construct overlaid ECDF plots for both partners on the same axes.

2. Construct a normal Q-Q plot for each partner separately.
3. Using the ECDF, determine at what delivery-time threshold Partner A and Partner B diverge most, and interpret this in operational terms.
4. Using the Q-Q plots, assess whether either partner's data supports a normal-distribution assumption; identify evidence of skew or outliers.
5. Recommend whether SLA limits should be set using mean $\pm$ standard deviation or empirical percentiles, with justification.

Key competency: *ECDF, Q-Q plot, distribution comparison, skewness, outliers*

Code:

```
library(ggplot2)

partner_a <- c(18, 19, 20, 20, 21, 21, 22, 22, 23, 23,
              24, 24, 25, 26, 27, 28, 30, 33, 38, 46)

partner_b <- c(20, 21, 22, 22, 23, 23, 23, 24, 24, 24,
              24, 25, 25, 25, 26, 26, 27, 27, 28, 29)

delivery_data <- data.frame(
  Partner = factor(rep(c("Partner A", "Partner B"), each = 20)),
  Delivery_Time = c(partner_a, partner_b)
)

ggplot(delivery_data,
  aes(x = Delivery_Time, color = Partner)) +
  stat_ecdf(geom = "step", linewidth = 1) +
  labs(
    title = "ECDF of Delivery Times",
    x = "Delivery Time (Hours)",
    y = "Cumulative Proportion" ) +
```

```

theme_minimal()

ggplot(subset(delivery_data, Partner == "Partner A"),
       aes(sample = Delivery_Time)) +
  stat_qq() +
  stat_qq_line() +
  labs(
    title = "Normal Q-Q Plot - Partner A",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_minimal()

ggplot(subset(delivery_data, Partner == "Partner B"),
       aes(sample = Delivery_Time)) +
  stat_qq() +
  stat_qq_line() +
  labs(
    title = "Normal Q-Q Plot - Partner B",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles") +
  theme_minimal()

aggregate(Delivery_Time ~ Partner,
          data = delivery_data,
          FUN = function(x)
            c(Mean = mean(x),
              Median = median(x),
              SD = sd(x)))

```

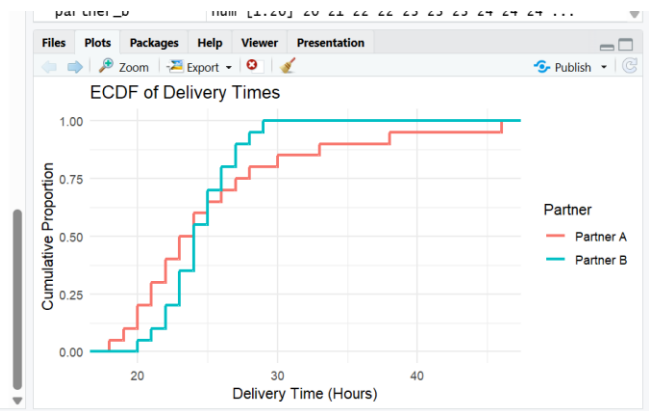
```
aggregate(Delivery_Time ~ Partner,
```

```
data = delivery_data,
```

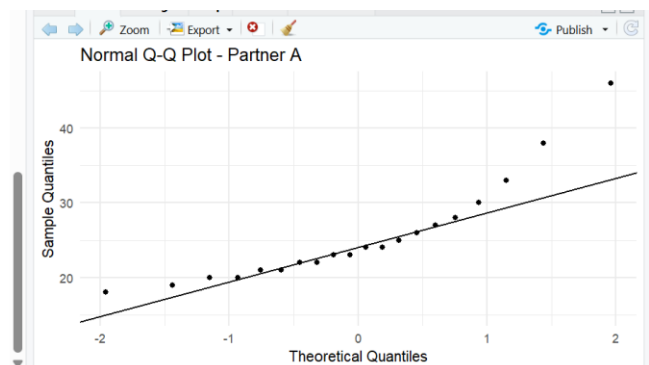
```
FUN = function(x)
```

```
quantile(x, probs = 0.95))
```

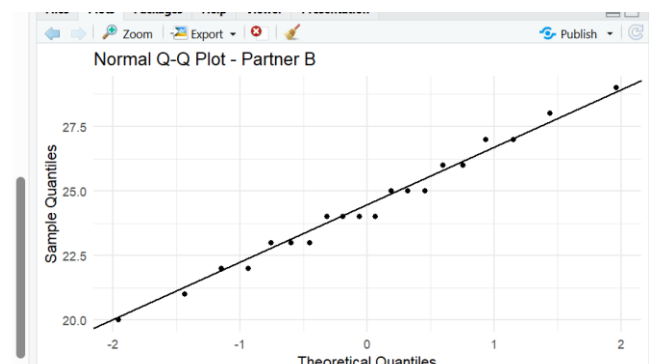
```
+ y ~ sample_quantiles
+ ) +
+ theme_minimal()
+
+ # Calculate summary statistics
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ c(Mean = mean(x),
+ Median = median(x),
+ SD = sd(x)))
+
+ Partner Delivery_Time.Mean Delivery_Time.Median Delivery_Time.SD
+ 1 Partner A 25.500000 23.500000 6.893933
+ 2 Partner B 24.400000 24.000000 2.326054
+
+ # Calculate empirical 95th percentile
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ quantile(x, probs = 0.95))
+
+ Partner Delivery_Time
+ 1 Partner A 38.40
+ 2 Partner B 28.05
```



```
+
+ # Calculate summary statistics
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ c(Mean = mean(x),
+ Median = median(x),
+ SD = sd(x)))
+
+ Partner Delivery_Time.Mean Delivery_Time.Median Delivery_Time.SD
+ 1 Partner A 25.500000 23.500000 6.893933
+ 2 Partner B 24.400000 24.000000 2.326054
+
+ # Calculate empirical 95th percentile
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ quantile(x, probs = 0.95))
+
+ Partner Delivery_Time
+ 1 Partner A 38.40
+ 2 Partner B 28.05
```



```
+
+ # Calculate summary statistics
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ c(Mean = mean(x),
+ Median = median(x),
+ SD = sd(x)))
+
+ Partner Delivery_Time.Mean Delivery_Time.Median Delivery_Time.SD
+ 1 Partner A 25.500000 23.500000 6.893933
+ 2 Partner B 24.400000 24.000000 2.326054
+
+ # Calculate empirical 95th percentile
+ aggregate(Delivery_Time ~ Partner,
+ data = delivery_data,
+ FUN = function(x)
+ quantile(x, probs = 0.95))
+
+ Partner Delivery_Time
+ 1 Partner A 38.40
+ 2 Partner B 28.05
```



- Partner A has lower delivery times for many shipments but has a long right tail, reaching 46 hours.
- Partner B has more tightly clustered delivery times, mainly between 20 and 29 hours.

- The largest operational divergence becomes noticeable in the upper delivery-time range, particularly around 29–30 hours and above. Partner B has completed essentially all deliveries by 29 hours, while Partner A still has a substantial fraction of deliveries extending beyond this threshold.
- The Q-Q plot for Partner A is expected to show strong deviation from the reference line in the upper tail because of values such as 38 and 46, indicating right skew/outliers.
- Partner B should be closer to a normal pattern, although its repeated central values may still produce some deviation.

SLA Recommendation

Use empirical percentiles, such as the 90th or 95th percentile, rather than $\text{Mean} \pm \text{SD}$, especially for Partner A.

Reason: Extreme delays make the mean and standard deviation less representative of actual service performance.

Potential Visualization Bias

Using only the mean could hide Partner A's rare but severe delays. Also, changing the ECDF axis range could make differences appear larger or smaller than they actually are.

Question 2 — Distribution Shape Across Many Groups using Bean Plots

Scenario: A hospital network measures patient waiting time (minutes) in five outpatient departments over a week. Administration wants to identify which departments have consistent waiting times and which have unpredictable, multi-modal patterns before reallocating staff.

Department	Waiting Times (minutes)
Cardiology	10, 12, 13, 14, 15, 15, 16, 17, 18, 20
Orthopedics	8, 9, 25, 26, 27, 9, 10, 28, 29, 8
Pediatrics	5, 6, 6, 7, 7, 8, 8, 9, 9, 10

Dermatology	15, 16, 17, 18, 19, 20, 21, 22, 23, 24
-------------	--

General Medicine	10, 40, 12, 38, 11, 41, 13, 39, 12, 40
------------------	--

Tasks:

1. Construct bean plots (or an equivalent density-plus-jitter visualization) comparing waiting-time distributions across all five departments on a shared vertical axis.
2. Identify which department(s) show bimodal or multi-modal patterns and explain what this suggests about underlying causes.
3. Compare the bean plot to a standard boxplot of the same data and state what the boxplot conceals.
4. Rank the departments by variability and by median waiting time separately, and explain why the two rankings differ.
5. Recommend one staffing or scheduling change based on the distribution shapes observed. **Key competency:** *Multiple distributions, vertical-axis visualization, bean plots, multi-modality, variability*

Code:

```
library(ggplot2)

department <- c(
  rep("Cardiology", 10),
  rep("Orthopedics", 10),
  rep("Pediatrics", 10),
  rep("Dermatology", 10),
  rep("General Medicine", 10))

waiting_time <- c(
  10, 12, 13, 14, 15, 15, 16, 17, 18, 20,
```

```
8, 9, 25, 26, 27, 9, 10, 28, 29, 8,  
5, 6, 6, 7, 7, 8, 8, 9, 9, 10,  
15, 16, 17, 18, 19, 20, 21, 22, 23, 24,  
10, 40, 12, 38, 11, 41, 13, 39, 12, 40)
```

```
hospital_data <- data.frame(  
  Department = department,  
  Waiting_Time = waiting_time)  
ggplot(hospital_data,  
  aes(x = Department, y = Waiting_Time)) +  
  geom_violin(trim = FALSE, alpha = 0.6) +  
  geom_jitter(width = 0.1, size = 2) +  
  stat_summary(  
    fun = median,  
    geom = "point",  
    size = 3  
  ) +  
  labs(  
    title = "Waiting Time Distribution Across Departments",  
    x = "Department",  
    y = "Waiting Time (Minutes)"  
  ) +  
  theme_minimal()  
ggplot(hospital_data,  
  aes(x = Department, y = Waiting_Time)) +  
  geom_boxplot() +  
  labs(  
    title = "Waiting Time Distribution Across Departments",  
    x = "Department",  
    y = "Waiting Time (Minutes)"  
  ) +  
  theme_minimal()
```

```

title = "Boxplot of Patient Waiting Times",

x = "Department",

y = "Waiting Time (Minutes)" ) +

theme_minimal()

summary_stats <- aggregate(

Waiting_Time ~ Department,

data = hospital_data,

FUN = function(x)

c(

Median = median(x),

SD = sd(x),

Range = max(x) - min(x)))

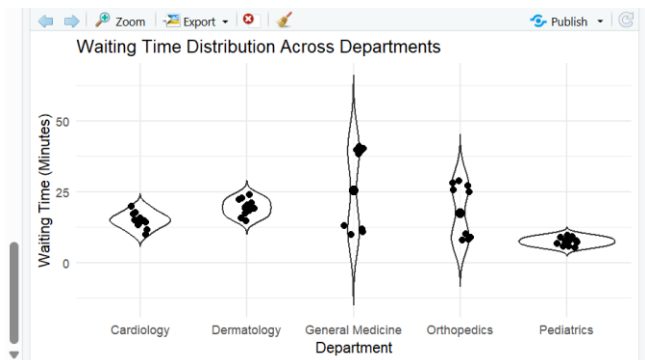
print(summary_stats)

```

```

> # 4. Median and Standard Deviation
> summary_stats <- aggregate(
+   Waiting_Time ~ Department,
+   data = hospital_data,
+   FUN = function(x)
+     c(
+       Median = median(x),
+       SD = sd(x),
+       Range = max(x) - min(x)
+     )
+ )
> print(summary_stats)
  Department Waiting_Time.Median Waiting_Time.SD Waiting_Time.Range
1   Cardiology      15.000000      2.943920      10.000000
2  Dermatology      19.500000      3.027650       9.000000
3 General Medicine      25.500000     14.796396     31.000000
4  Orthopedics      17.500000      9.666092     21.000000
5   Pediatrics       7.500000      1.581139       5.000000

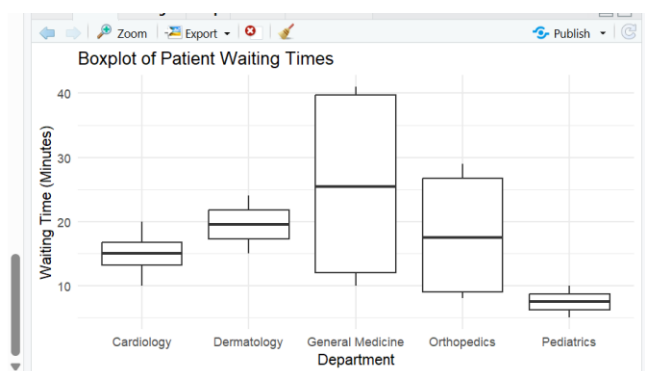
```



```

> # 4. Median and Standard Deviation
> summary_stats <- aggregate(
+   Waiting_Time ~ Department,
+   data = hospital_data,
+   FUN = function(x)
+     c(
+       Median = median(x),
+       SD = sd(x),
+       Range = max(x) - min(x)
+     )
+ )
> print(summary_stats)
  Department Waiting_Time.Median Waiting_Time.SD Waiting_Time.Range
1   Cardiology      15.000000      2.943920      10.000000
2  Dermatology      19.500000      3.027650       9.000000
3 General Medicine      25.500000     14.796396     31.000000
4  Orthopedics      17.500000      9.666092     21.000000
5   Pediatrics       7.500000      1.581139       5.000000

```



Bimodal Departments

Orthopedics shows a clear bimodal pattern:

- One group is around **8–10 minutes**
- Another group is around **25–29 minutes**

This may indicate two different types of patients, appointment schedules, or service processes.

General Medicine also shows a strong two-group pattern:

- One group around **10–13 minutes**
- Another around **38–41 minutes**

This suggests highly inconsistent scheduling or two different patient-processing conditions.

What the Boxplot Conceals

A boxplot shows:

- Median
- Quartiles
- Range
- Potential outliers

However, it may conceal the fact that Orthopedics and General Medicine contain **two distinct clusters of waiting times**.

Variability Ranking

From lower to higher variability, approximately:

1. Pediatrics
2. Cardiology
3. Dermatology
4. Orthopedics
5. General Medical

Question 3 — Choosing an Honest Proportion Chart

Scenario: A retail chain surveys 300 customers across three store formats about preferred payment method. Senior management wants a dashboard chart that allows accurate comparison of payment-method share across formats without visually exaggerating small gaps.

Store Format	Cash (%)	Card (%)	UPI/Wallet (%)
Hypermarket	18	34	48
Supermarket	22	30	48
Convenience	35	25	40

Tasks:

1. Construct a side-by-side (grouped) bar chart and a 100% stacked bar chart for the three store formats.
2. Construct three separate pie charts (one per format) representing the same data.
3. Compare all three visualizations for how easily a viewer can judge which format has the highest UPI/Wallet share.
4. Explain, with reference to arc-length versus length perception, why the pie charts are harder to compare accurately than the bar charts.
5. Identify one design decision (e.g., axis truncation, 3D effect, category ordering) that could be used to make the Convenience format's cash usage appear more dramatic than it is, and explain why it would mislead.

Key competency: Proportions, side-by-side bars, stacked bars, pie charts, misleading design

Code:

```
payment <- matrix(c(
  18,22,35,
  34,30,25,
```

```

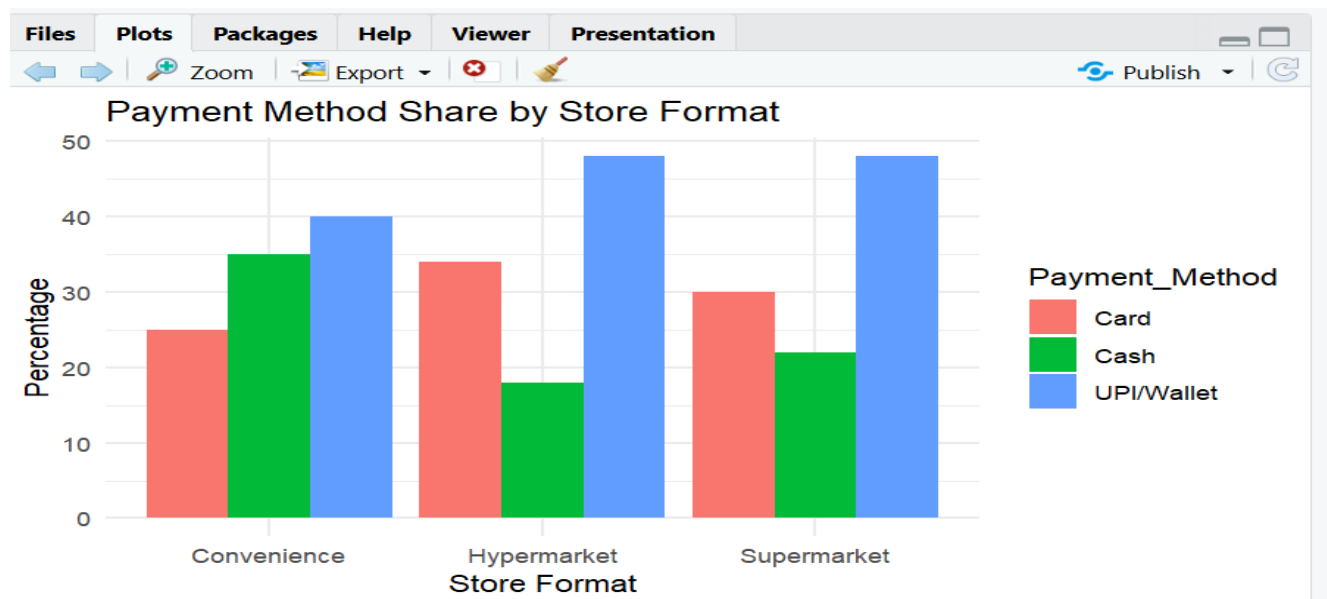
48,48,40),
nrow = 3,
byrow = TRUE)
rownames(payment) <- c("Cash", "Card", "UPI/Wallet")
colnames(payment) <- c("Hypermarket",
                      "Supermarket",
                      "Convenience")
barplot(payment,
        beside = TRUE,
        col = c("orange", "skyblue", "lightgreen"),
        main = "Payment Method by Store Format",
        xlab = "Store Format",
        ylab = "Percentage (%)",
        legend.text = rownames(payment))
barplot(payment,
        beside = FALSE,
        col = c("orange", "skyblue", "lightgreen"),
        main = "100% Stacked Bar Chart",
        xlab = "Store Format",
        ylab = "Percentage (%)",
        legend.text = rownames(payment))
pie(payment[,1],
    labels = paste(rownames(payment),
                  payment[,1], "%"),
    main = "Hypermarket")

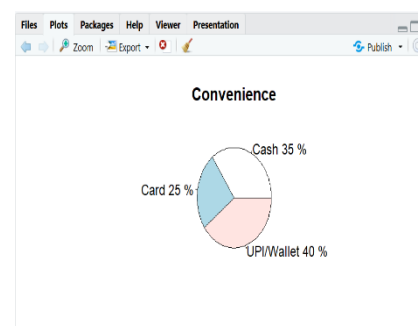
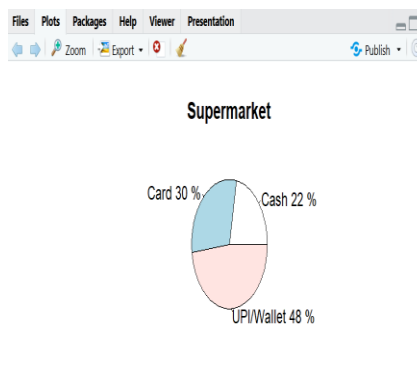
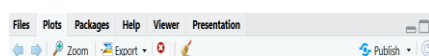
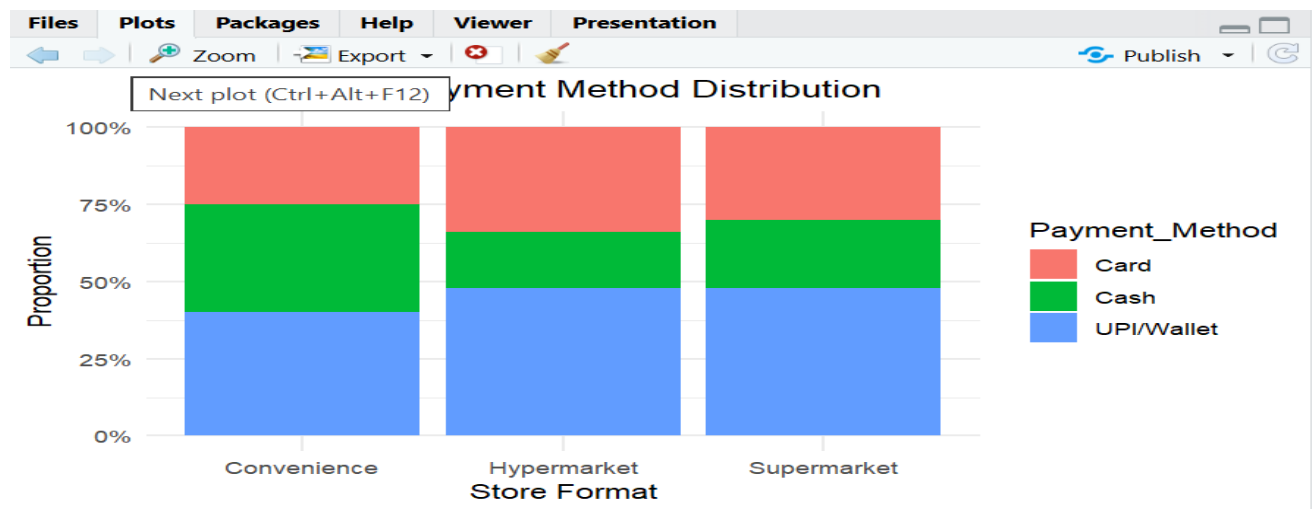
```

```

pie(payment[,2],
     labels = paste(rownames(payment),
                     payment[,2], "%"),
     main = "Supermarket")
pie(payment[,3],
     labels = paste(rownames(payment),
                     payment[,3], "%"),
     main = "Convenience")
upi <- payment["UPI/Wallet",]
cat("UPI/Wallet Percentages:\n")
print(upi)
cat("\nHighest UPI/Wallet Share:\n")
print(names(which.max(upi)))

```





Highest UPI/Wallet Share

- Hypermarket = **48%**
- Supermarket = **48%**
- Convenience = **40%**

The grouped bar chart makes this comparison the easiest because all UPI/Wallet bars begin from the same baseline.

The 100% stacked bar chart also communicates proportions well, but segments not starting from zero are harder to compare.

Why Pie Charts Are Harder

Humans compare **length and position** more accurately than **angles and arc lengths**.

Therefore:

- Bar charts → easier comparison
- Pie charts → harder comparison between separate charts

Misleading Design Example

A truncated vertical axis starting at **30% instead of 0%** could make Convenience cash usage at **35%** look dramatically larger than Hypermarket's **18%**.

Question 4 — Nested Proportions in a Household Budget

Scenario: A financial-planning app aggregates the average monthly household budget (₹60,000) across spending categories and subcategories for a client segment. The product team wants a dashboard visual that shows both overall category share and subcategory composition within each category.

Category	Subcategory	Amount (₹)
Housing	Rent/EMI	18000
Housing	Utilities	4000

Housing	Maintenance	2000
Food	Groceries	9000
Food	Dining Out	3000
Transport	Fuel	3500
Transport	Public Transit	1500
Savings	Investments	10000
Savings	Emergency Fund	4000

Discretionary	Entertainment	3000
Discretionary	Shopping	2000

Tasks:

1. Construct a treemap showing category-level and subcategory-level allocation of the budget.
2. Construct a sunburst chart representing the same two-level hierarchy.
3. Calculate the percentage of total budget consumed by the Housing category and by its largest subcategory.
4. Compare the treemap and sunburst chart for how easily a viewer can compare two subcategories that belong to different categories.
5. Describe one scenario in which the area-encoding of the treemap could cause a viewer to misjudge the relative size of two categories.

Key competency: *Treemaps, sunburst charts, nested proportions, area encoding*

Code:

```
library(plotly)

data <- data.frame(
  Category = c("Housing","Housing","Housing",
    "Food","Food",
    "Transport","Transport",
    "Savings","Savings",
    "Discretionary","Discretionary"),
  Subcategory = c("Rent/EMI","Utilities","Maintenance",
    "Groceries","Dining Out",
    "Fuel","Public Transit",
```

```

      "Investments", "Emergency Fund",
      "Entertainment", "Shopping"),
Amount = c(18000, 4000, 2000,
           9000, 3000,
           3500, 1500,
           10000, 4000,
           3000, 2000))

totals <- aggregate(Amount ~ Category, data, sum)
labels <- c("Total Budget", totals$Category, data$Subcategory)
parents <- c("",
             rep("Total Budget", nrow(totals)),
             data$Category)

values <- c(60000, totals$Amount, data$Amount)

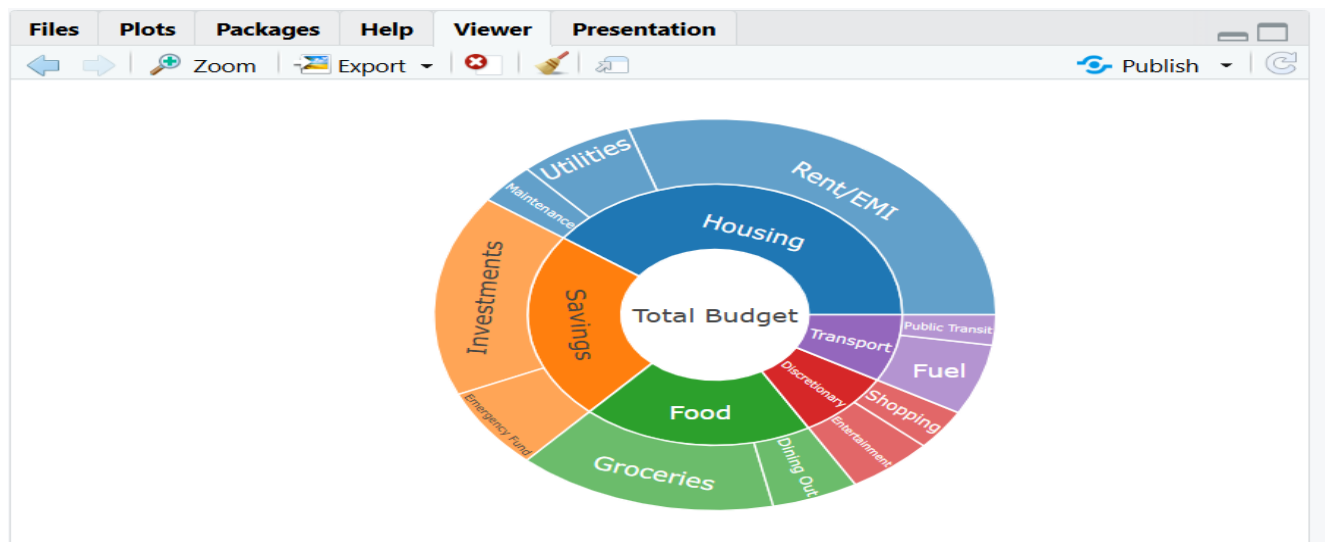
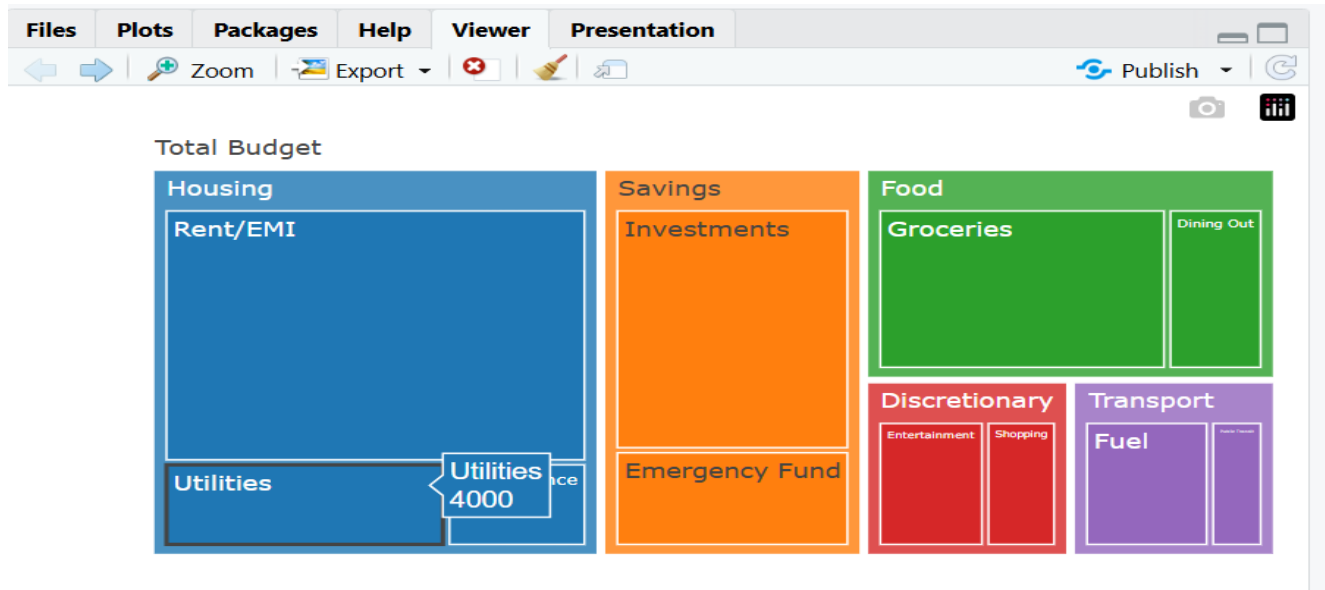
plot_ly(
  labels = labels,
  parents = parents,
  values = values,
  type = "treemap",
  branchvalues = "total")

plot_ly(
  labels = labels,
  parents = parents,
  values = values,
  type = "sunburst",
  branchvalues = "total")

housing <- sum(data$Amount[data$Category == "Housing"])

```

```
rent <- data$Amount[data$Subcategory == "Rent/EMI"]
cat("Housing Percentage =", housing / 60000 * 100, "%\n")
cat("Rent/EMI Percentage =", rent / 60000 * 100, "%\n")
```



Treemap

The treemap is useful for comparing the **relative sizes** of budget components because larger amounts occupy larger rectangular areas.

Sunburst

The sunburst clearly shows the **hierarchical relationship**:

Total Budget → Category → Subcategory

Comparing Across Categories

A treemap is generally better for comparing subcategories from different categories because viewers can compare rectangular areas more directly.

Potential Bias

Humans may find it difficult to accurately compare areas of rectangles with very different shapes.

For example, two categories with similar amounts may look noticeably different if one rectangle is long and narrow while another is more square-shaped.

Question 5 — Flow Analysis using Sankey Diagram and Parallel Sets

Scenario: An online learning platform tracks 250 students from the course they enrolled in, through the device used to study, to their final completion status. The academic team wants to understand dominant enrollment-to-outcome pathways and spot problem areas.

Course	Device	Completion Status	Students
Data Science	Mobile	Completed	20
Data Science	Mobile	Dropped	25
Data Science	Laptop	Completed	45
Data Science	Laptop	Dropped	10
Web Development	Mobile	Completed	15
Web Development	Mobile	Dropped	20
Web Development	Laptop	Completed	35
Web Development	Laptop	Dropped	10
Digital Marketing	Mobile	Completed	10

Digital Marketing	Mobile	Dropped	15
Digital Marketing	Laptop	Completed	20
Digital Marketing	Laptop	Dropped	25

Tasks:

1. Construct a Sankey diagram showing the flow Course → Device → Completion Status.
2. Construct a parallel-sets (alluvial) representation of the same categorical flows.
3. Identify the three largest flows in the dataset and calculate each as a percentage of all 250 students.
4. Determine which course has the weakest completion outcome and identify at what stage in the flow the drop-off is most visible.
5. Explain one way that inconsistent node ordering or unscaled link widths could mislead a viewer comparing courses.

Key competency: *Sankey diagrams, parallel sets, flow proportions, categorical relationships*

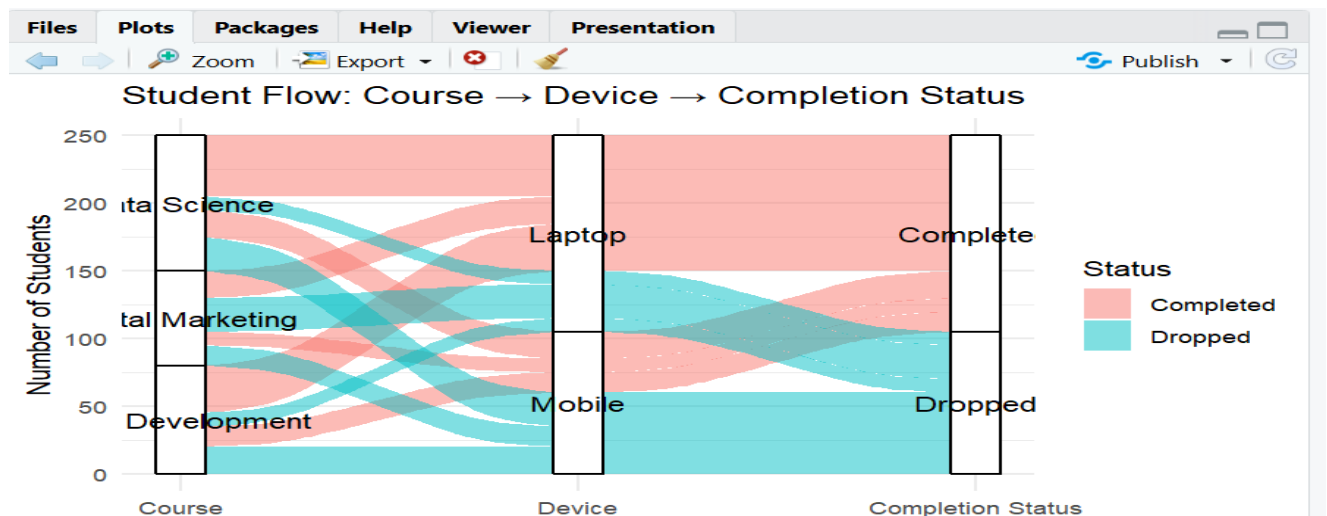
Code:

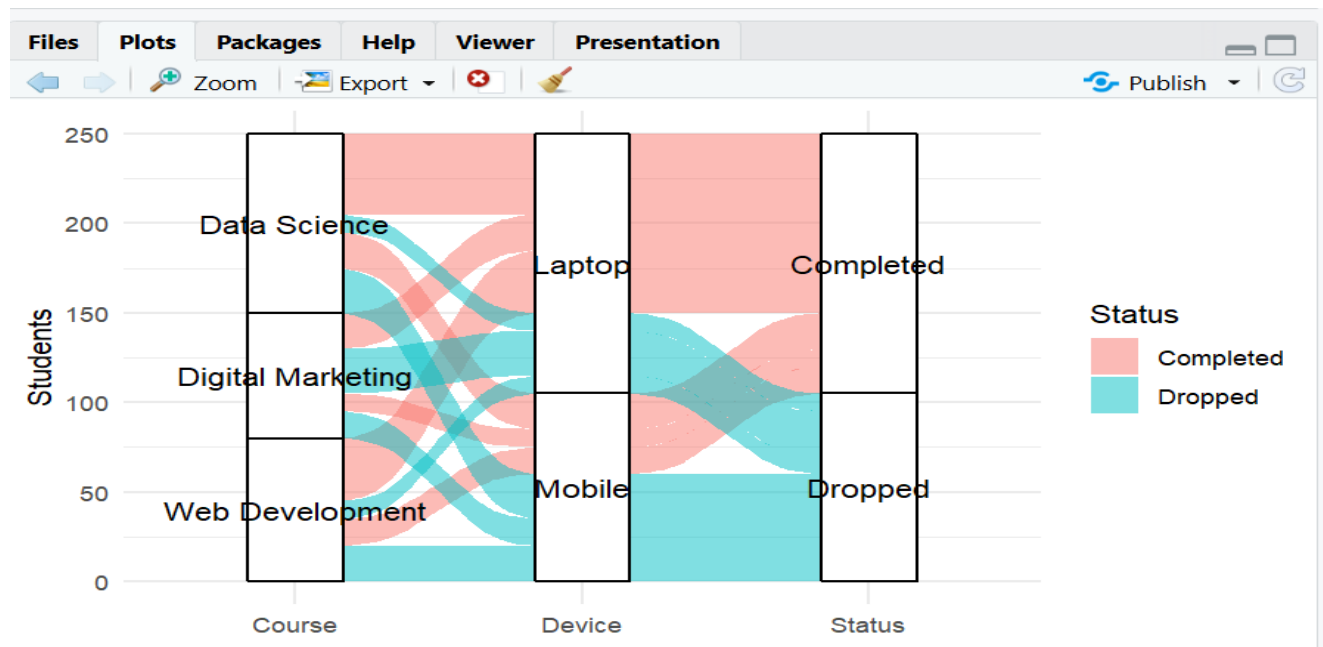
```
library(ggplot2)
library(ggalluvial)
ggplot(
  flow_data,
  aes(axis1 = Course,
      axis2 = Device,
      axis3 = Status,
      y = Students)) +
  geom_alluvium(
    aes(fill = Status),
```

```

width = 1/12) +
geom_stratum(
  width = 1/8) +
geom_text(
  stat = "stratum",
  aes(label = after_stat(stratum))) +
scale_x_discrete(
  limits = c(
    "Course",
    "Device",
    "Completion Status" ),
  expand = c(.05, .05)) +
labs(
  title = "Student Flow: Course → Device → Completion Status",
  y = "Number of Students") +
theme_minimal()

```





Completion Rates

Data Science

- Completed = $20 + 45 = 65$
- Dropped = $25 + 10 = 35$
- Total = 100
- Completion Rate = **65%**

Web Development

- Completed = $15 + 35 = 50$
- Dropped = $20 + 10 = 30$
- Total = 80
- Completion Rate = **62.5%**

Digital Marketing

- Completed = $10 + 20 = 30$
- Dropped = $15 + 25 = 40$
- Total = 70
- Completion Rate = **42.86%**

Digital Marketing → Laptop → Dropped

flow containing **25 students**.

Important Dataset Observation

The listed values actually sum to:

$$100 + 80 + 70 = 250$$

So the dataset correctly contains **250 students**.

Suggested Evaluation Rubric — Data Analysis Project

Criterion	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Data preparation	Correct and complete	Minor issue	Several issues	Incorrect/incomplete
Visualization choice	Highly appropriate and justified	Appropriate	Partially appropriate	Poor choice
Technical execution	Accurate and clear	Mostly accurate	Some errors	Major errors
Interpretation	Insightful and evidence based	Correct	Basic	Incorrect/unsupported
Misleading-design analysis	Bias clearly identified and explained	Bias identified	Limited explanation	Not identified

Suggested marks: 20 marks per question; 100 marks total. Level: Competitive / advanced undergraduate.